

Code-review по ДЗ №1

Студент №1 - uc7RbPAkLf

(<https://disk.360.yandex.com/d/X5OwsbVTAxkCoQ>)

1. Все ли, по вашему мнению, ключевые сущности предметной области выявлены и присутствуют?

В целом да. Выделены базовые сущности:

- IAlive
- IInventory
- Animal + наследники (Herbo, Predator, Monkey, Rabbit, Tiger, Wolf)
- Thing + Table, Computer
- Zoo
- VeterinaryClinic

То есть все обязательные сущности из тз присутствуют.

Структура в точности соответствует заданию.

2. Если какие-то из ключевых сущностей отсутствуют, то какие?

Отсутствующих критичных сущностей я не увидел. Все ок

3. Насколько автор придерживался условий задания (структура, классы, интерфейсы, связи)? Если автор где-то отклонился от условия / расширил проект, отразилось ли это на качестве программы и как?

Структура, классы, интерфейсы, связи:

Автор следует требованиям:

- Есть две ветки иерархии: животные и вещи.
- Интерфейсы IAlive и IInventory реализованы корректно.
- Ассоциация Zoo -> VeterinaryClinic выполнена правильно: зависимость внедряется через конструктор.
- Логика проверки здоровья перед добавлением животного реализована.
- Все животные и вещи учитываются в инвентаре.
- Формирование списка животных с добротой > 5 выполнено.

Отклонения / расширения:

Критичных отклонений нет, но некоторые моменты выглядят упрощенными:

- Валидация входных данных минимальная. Например, KindnessLevel не проверяется на [1 .. 10], хотя в тз шкала 1-10.
- Zoo хранит Animal и Thing в отдельных списках, хотя мог бы хранить их единообразно как IInventory - но это вроде не нарушение.

В целом отклонения не ухудшают, но немного упрощают поведение.

4. *Адекватен ли уровень абстракции условию задачи?*

Да - абстракции распределены корректно:

- Animal - абстрактная основа.
- Herbo и Predator логично разделяют типы животных.
- Конкретные виды - простые подклассы без лишней логики (Monkey, Tiger, Wolf и тд).
- VeterinaryClinic содержит только ответственность медицинской проверки.

Уровень абстракции в целом соответствует тому, что ожидалось.

5. *Применены ли в задании принципы ООП и SOLID?*

Да, применены.

ООП: наследование, абстракции, интерфейсы, полиморфизм.

SOLID: четкое разделение ответственности между Animal, Zoo, Clinic, интерфейсами.

6. *Выделите, за счет чего в программе реализуются принципы SOLID? Приведите 2-3 принципа и пример их реализации в проекте автора*

Single responsibility:

VeterinaryClinic отвечает только за проверку здоровья.

Zoo отвечает за управление коллекциями, инвентарем, отчетами.

Open/closed:

Добавление новых животных не требует изменения существующих классов: достаточно создать новый наследник Animal.

Dependency inversion:

Zoo получает IVeterinaryClinic через конструктор - зависимость от абстракции, не от реализации.

7. *Выделите, за счет чего в программе реализуются принципы ООП? Приведите 2-3 принципа и пример их реализации в проекте автора*

Наследование:

Herbo : Animal

Predator : Animal

Monkey : Herbo

Tiger : Predator

и т.д.

Полиморфизм:

Методы Zoo работают с IAlive / IInventory / Animal абстрактно

Абстракция:

Animal - четкая абстракция живых инвентаризуемых сущностей.

8. *Использован ли DI-контейнер в проекте? Как вам кажется, корректно ли он использовался?*

DI реализован через внедрение зависимостей в конструктор, но DI-контейнера как такового нет.

То есть DIP формально вроде и соблюден, но контейнер отсутствует.

С точки зрения тз это -1.5 балла и -0.5 как за соблюдение принципа, все же дз называлось “Закрепление принципов SOLID, внедрения зависимостей средствами DI-контейнера.”

9. *Что в работе вам понравилось? Какие идеи коллеги вы бы хотели встроить в свои собственные работы в дальнейшем?*

- Структура чистая и читаемая.
- Стилль кода один (XML-комментарии, аккуратные файлы).
- Хорошая реализация абстракций для животных.
- Лаконичный и грамотный Zoo.
- Корректное использование интерфейсов без избыточных зависимостей.

10. *Что вы можете посоветовать своему коллеге для улучшения качества работы/для развития навыков проектирования?*

С точки зрения тз:

- Добавить настоящую валидацию входных параметров:

kindnessLevel должен быть 1..10

food > 0

number > 0

валидировать наличие имени и тп

- Добавить DI-контейнер
- Унифицировать хранение инвентаря

Можно хранить все как List<Inventory> вместо двух списков.

11. *Какую оценку вы бы поставили за работу? Аргументируйте*

Я бы поставил 7.5 / 10

- Нет DI-контейнера - минус 1.5 и минус 0.5 как за соблюдение solid
- Неполная валидация - минус 1 (этому уделяется немалое внимание в тз)

Все остальное в норме

Студент №2 - 6s9bDymr8A

(<https://disk.360.yandex.com/d/flfv5l9sFgpbCA>)

1. Все ли, по вашему мнению, ключевые сущности предметной области выявлены и присутствуют?

Да, все ключевые сущности из тз реализованы:

- Абстракции: Animal, Herbivore, Predator, Thing, InventoryItem, Employee.
- Конкретные животные: Rabbit, Monkey, Tiger, Wolf.
- Конкретные вещи: Table, Computer.
- Интерфейсы: IFeedable (аналог IAlive), IInventory, IInventoryPrinter, IHealthCheck.
- Сервисные классы: Zoo, VeterinaryClinic, Veterinarian, App, SpectreConsolePrinter.
- DI-конфигурация: ServiceConfiguration, точка входа Program.

Требования к учету животных, расчету суммарной еды, выделению контактного зоопарка и учету инвентаря соблюдены.

2. Если какие-то из ключевых сущностей отсутствуют, то какие?

Отсутствующих критичных сущностей я не увидел. Вместо интерфейса IAlive введен IFeedable с нужным свойством FoodPerDay, что соответствует смыслу тз.

3. Насколько автор придерживался условий задания (структура, классы, интерфейсы, связи)? Если автор где-то отклонился от условия / расширил проект, отразилось ли это на качестве программы и как?

Структура, классы, интерфейсы, связи:

Придерживается очень хорошо:

- Есть отдельные иерархии животных и вещей.
- Животные реализуют и интерфейс IFeedable, и инвентарный (IInventory) через базовый InventoryItem.
- Вещи реализуют только IInventory.
- Класс Zoo агрегирует IInventory и использует IHealthCheck для проверки здоровья при приеме животных.
- VeterinaryClinic реализует IHealthCheck и хранит список ветеринаров.
- Выделен отдельный интерфейс IInventoryPrinter и реализация SpectreConsolePrinter - все взаимодействие с консолью вынесено туда.
- Отдельный класс App отвечает за диалог с пользователем и использование сервиса Zoo.

Отклонения / расширения:

Вместо предложенного IAlive студент использует IFeedable, в целом логично.

Добавлены абстракции InventoryItem, Employee, Veterinarian, а также интерфейс для печати IInventoryPrinter - это расширяет архитектуру и повышает ее чистоту.

Используется библиотека Spectre.Console для красивого вывода.

Все отклонения от тз только улучшают качество решения.

4. *Адекватен ли уровень абстракции условию задачи?*

Да, уровень абстракции очень удачный:

- Базовые абстрактные классы (Animal, Herbivore, Predator, Thing, InventoryItem) аккуратно отделяют общие свойства от частных.
- Zoo - высокоуровневый сервис предметной области (работа с животными и инвентарем), не содержит логики UI.
- VeterinaryClinic и Veterinarian отвечают за медицинскую часть.
- App - тонкая оболочка над предметной моделью, реализующая меню и ввод/вывод.
- InventoryPrinter отделяет представление данных (консоль) от доменной логики.

Модель не перегружена, но расширяема

5. *Применены ли в задании принципы ООП и SOLID?*

Да, и довольно аккуратно:

ООП: используется наследование, абстрактные классы, интерфейсы, полиморфизм по базовым типам (Animal, Herbivore, Inventory).

SOLID: явно просматриваются все принципы, особенно в связке Zoo - IHealthCheck - VeterinaryClinic и в использовании DI-контейнера.

6. *Выделите, за счет чего в программе реализуются принципы SOLID? Приведите 2-3 принципа и пример их реализации в проекте автора*

Single responsibility:

- VeterinaryClinic отвечает только за проверку здоровья животных.
- Zoo отвечает за управление коллекциями животных и вещей, подсчет еды и формирование отчетов.
- App отвечает за сценарий взаимодействия с пользователем (меню, ввод/вывод).

Open/closed:

Добавление новых животных и вещей не требует изменения существующих классов: достаточно создать нового наследника Animal или Thing, и логика Zoo продолжит работать без изменений.

Dependency inversion:

Zoo зависит от интерфейса IHealthCheck, а не от конкретного класса VeterinaryClinic; реализация подставляется через DI-контейнер.

7. *Выделите, за счет чего в программе реализуются принципы ООП? Приведите 2-3 принципа и пример их реализации в проекте автора*

Наследование:

Herbivore : Animal

Predator : Animal
Rabbit, Monkey : Herbivore
Tiger, Wolf : Predator
Table, Computer : Thing

Полиморфизм:

Zoo работает с коллекцией `List<IInventory>`, где могут находиться и животные, и вещи, а также метод `Inspect` в `IHealthCheck` принимает `Animal` и одинаково обрабатывает любые конкретные виды животных.

Абстракция:

`Animal`, `Herbivore`, `Predator` и `IInventoryItem` - четкие абстракции, в которых сосредоточены общие свойства и поведение. Конкретные классы-наследники переопределяют только специфические детали.

8. *Использован ли DI-контейнер в проекте? Как вам кажется, корректно ли он использовался?*

Да, корректно.

Использование DI-контейнера корректное и аккуратное: разделена конфигурация, использованы подходящие жизненные циклы. Есть даже отдельные тесты `DIContainerTests`, проверяющие корректную регистрацию.

9. *Что в работе вам понравилось? Какие идеи коллеги вы бы хотели встроить в свои собственные работы в дальнейшем?*

Что понравилось:

- Четкая архитектура с разделением домена / инфраструктуры / UI.
- Аккуратное использование `Spectre.Console` - приложение выглядит живым.
- Хорошо продуманная иерархия абстракций (`IInventoryItem`, `Herbivore`, `Predator`).
- Наличие полноценного проекта юнит-тестов (`ZooTests`, `VeterinarianTests`, `AppTests`, `ConsolePrinterTests`, `DIContainerTests`, `EmployeeTests`, `ProgramTests`).

Что бы я хотел позаимствовать:

- Подход с отдельным `IInventoryPrinter` и красивым выводом через `Spectre.Console`.
- Вынесение DI-конфигурации в отдельный статический класс-расширение.

10. *Что вы можете посоветовать своему коллеге для улучшения качества работы/для развития навыков проектирования?*

С точки зрения тз:

- Та же проблема - нужно добавить валидацию входных параметров:

kindnessLevel должен быть 1..10

food > 0

number > 0

корректность числовых вводов в App

валидировать наличие имени и тп

Все остальное на высоте.

11. *Какую оценку вы бы поставили за работу? Аргументируйте*

Я бы поставил 9.5 / 10

- Неполная валидация - минус 0.5

Все остальное почти идеально.